

Week 2: QML for Feature Extraction (OCR Stage)

Hybrid QML and Quantum String Algorithms for Document Extraction

Pushkar Kumar • Mentor: Potluri Krishna Priyatham

Deliverable as specified in the project brief: Simulation results of the quantum feature extraction layer.

1 Objective

Build a quantum neural network component for character and layout detection: design a quanvolutional layer to identify document features, and train a variational quantum circuit for basic character recognition.

2 The Quanvolutional Filter

A quanvolutional layer is the quantum analogue of a convolution. A small window slides over the image; at each position the patch is encoded into a quantum state, transformed by a circuit, and measured.

The filter is a fixed 4-qubit circuit: two layers of R_Y rotations with a ring of CX gates between them, angles drawn once from a seeded generator and held constant. This is the canonical *untrained random* formulation of Henderson et al.

Two architectural points determine whether it does anything useful. Rotations alone would be useless—without the CX gates the circuit acts independently on each qubit, making the output a deterministic function of each pixel separately, with no interaction between pixels, which is the entire purpose of a convolution. And the ring closure (last qubit back to qubit 0) allows information to reach every qubit within one layer, where a plain chain would leave qubit 0 unable to influence qubit $n - 1$ until the second layer.

2.1 Encoding

Each pixel value $v \in [0, 255]$ becomes a rotation angle $\theta = \pi v / 255$ applied to its own qubit:

$$R_Y(\theta)|0\rangle = \cos(\theta/2)|0\rangle + \sin(\theta/2)|1\rangle \quad (1)$$

A dark pixel remains near $|0\rangle$; a bright one rotates toward $|1\rangle$. Because each qubit is rotated independently, the four-qubit input is a **product state** with no entanglement. All entanglement in the output originates in the filter's CX gates.

3 Making Simulation Tractable

A direct implementation submits one simulator job per patch. At 16 patches per character and roughly 6,000 characters, that is approximately 96,000 jobs per pass over the dataset—several hours, which makes iteration impossible.

Because the filter is *fixed*, its unitary U is identical for every patch. It is extracted once and applied as a batched matrix multiplication:

$$|\psi_{\text{out}}\rangle = U |\psi_{\text{in}}\rangle \quad (2)$$

This is not an approximation. It is the same linear algebra the simulator performs, without per-job overhead and without shot noise. The claim was verified rather than asserted: agreement with the shot-based implementation was measured at a maximum deviation of **0.0028** across random patches at 200,000 shots, which is within sampling error. Processing time fell from hours to **0.15 seconds** for **2,000 characters**.

4 Readout Design

The obvious readout is per-qubit $P(\text{measure } 1)$: four values per patch.

But a 4-qubit circuit produces a distribution over 16 outcomes, and keeping only four marginals discards every *correlation* the entangling layer created—precisely the component a classical product-state model could not have produced. Adding pairwise correlations

$$\langle Z_i Z_j \rangle = \sum_k P(k) z_i(k) z_j(k) \quad (3)$$

recovers six additional values per patch, taking the readout from 4 to 10 values and the per-character feature vector from 64 to 160.

Table 1: Effect of readout design, all tiers pooled, matched classifier.

Readout	Feature dimension	Accuracy
Marginals only	64	86.7%
Marginals + $\langle Z_i Z_j \rangle$	160	92.2%

The 5.5-point gain from readout design alone exceeds any difference measured between the quantum and classical filters. For quanvolutional approaches generally, how much of the output distribution is retained appears to matter more than which circuit produced it.

5 Comparison Against Classical Baselines

The comparison holds feature dimensionality fixed. Without that, a win would be indistinguishable from simply granting one method a wider representation. The classical control uses ten random filters over the same 2×2 patches at the same stride, producing exactly 160 features.

Table 2: Character accuracy, all tiers pooled.

Extractor	Dimension	Accuracy
Quanvolutional, marginals	64	86.7%
Quanvolutional, + $\langle Z_i Z_j \rangle$	160	92.2%
Classical convolution, dimension-matched	160	91.9%
Raw pixels, no feature extraction	64	93.6%

Repeating over 30 paired train–test splits establishes which differences are real: the quantum-versus-classical gap is not distinguishable from noise ($p \approx 0.2$ – 0.3 , the quantum layer winning approximately half the splits), while the raw-pixel lead holds in all 30 ($p < 10^{-10}$).

The quantum layer does not beat the classical baselines. At matched dimensionality it ties a random classical convolution, and both lose to raw pixels at *lower* dimensionality. The relevant comparison is therefore not quantum versus classical but untrained random feature map versus no feature map: at 8×8 the input is already close to information-minimal, so any fixed random projection discards more than it contributes. That the classical control fails identically is what identifies the cause as the untrained random character of the filter rather than anything quantum.

6 Variational Circuit

The brief specified a *variational* circuit, so the filter’s eight R_Y angles were optimised against classification loss by gradient-free Powell search, starting from the untrained baseline’s exact angle vector so that any improvement is attributable to optimisation alone. The classical control was trained identically—training only the quantum side would have inverted the fairness the untrained comparison was designed to preserve.

Table 3: Variational training on a sealed test partition of 1,807 crops.

Configuration	Trainable parameters	Accuracy	Δ
Quantum, untrained	—	91.7%	—
Quantum, trained	8	90.6%	−1.1 pt
Classical, untrained	—	92.0%	—
Classical, trained	40	92.1%	+0.1 pt
Raw pixels	—	93.6%	—

Training does not close the gap, which strengthens rather than weakens the preceding section: the deficit is not caused by the filters being untrained, because training them does not help.

6.1 A Correction Worth Recording

An earlier version of this experiment reported a +1.4-point gain for the trained quantum filter, bringing it level with raw pixels. That result was wrong.

The angle search had been run over the full dataset, and the final evaluation then used a fresh random split of that *same* dataset—so the angles had been selected using rows that later appeared in the test set. Re-running with a sealed 30% partition that the search never sees reduced the gain from +1.4 points to approximately zero.

The inflated figure is recorded here rather than discarded, because the difference between the two is the difference between a false headline claim and the actual finding.

6.2 One Property That Survives

The quantum filter produces its 160-dimensional feature space from **8 parameters** against the classical control’s **40**. At the untrained baseline the two perform equivalently, so that 5× parameter economy is real and costs nothing there. It does not survive this training protocol, and should not be reported without that qualification.

7 Deliverable Status

Complete. Quanvolutional layer implemented and verified against the reference simulator; read-out design measured; dimension-matched classical comparison performed with significance testing; variational training implemented for both quantum and classical filters.

Artifacts: `integration/features.py`, `track_a_vision/quanvolutional/train_filter.py`, `notebooks/02_quanvolutional_features.ipynb`, `benchmarking/seed_sweep.py`